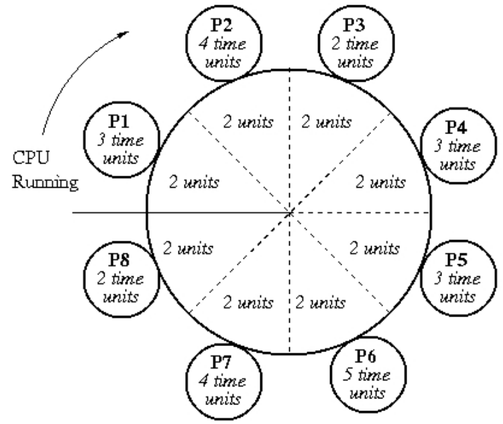
Universidad Nacional de Colombia
Estructura de Datos - 2016699

Planificación de Procesos Round Robin

Presentado por:

- Alejandro Borda
- Esteban Gaona
- Andrea Noreña
- Cristian Méndez



Contenido

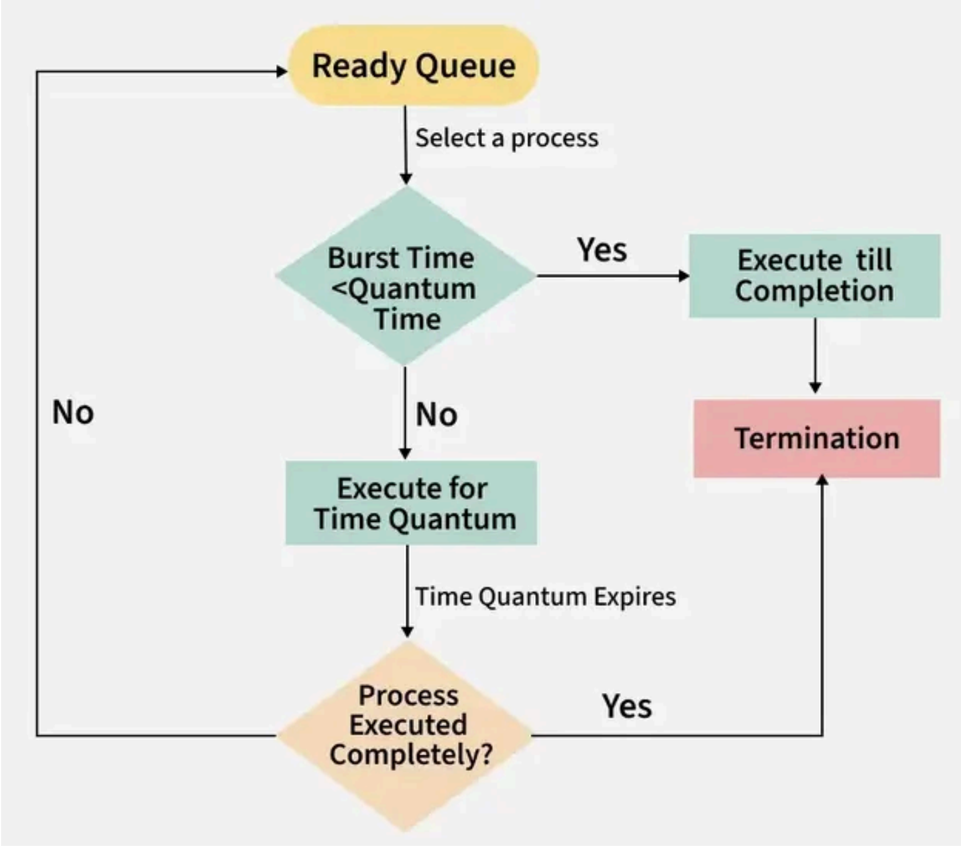
1. Introducción
2. Animación
3. Casos Borde
4. Complejidad
5. Créditos bibliográficos

Introducción

Round-robin scheduling

Algoritmo de planificación preventivo (*pre-emptive)* diseñado para asignar porciones de tiempo equitativas (*quantum o time slice*) a los procesos en una cola (*Ready Queue)*.

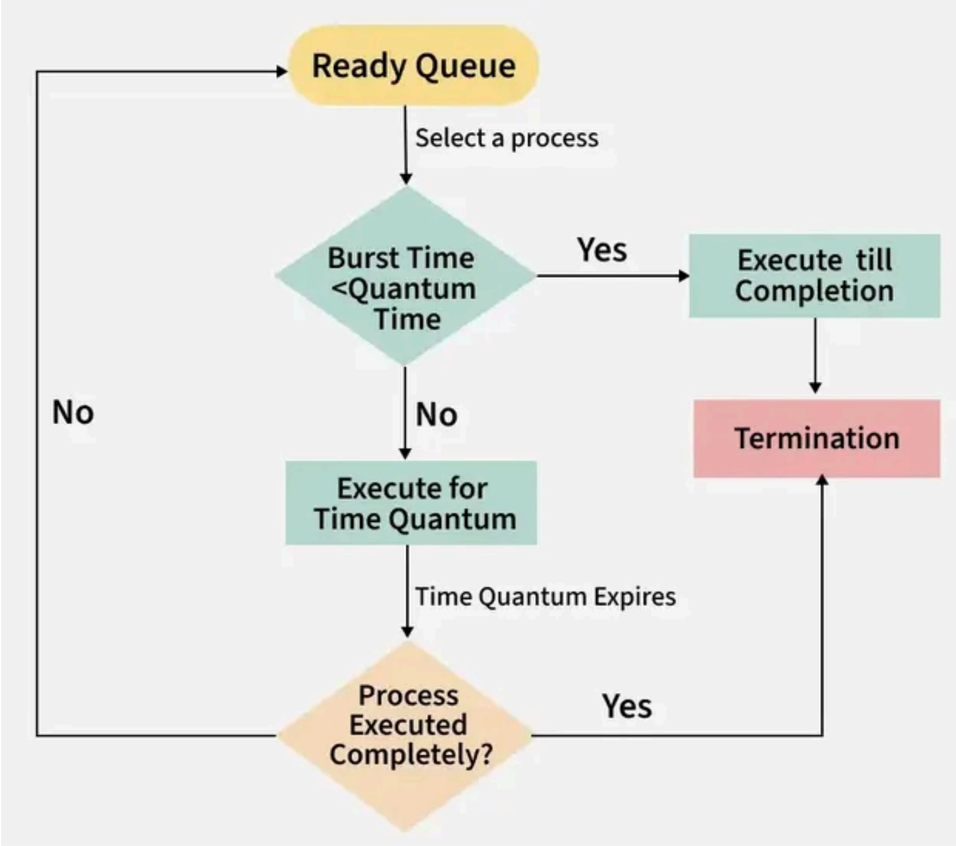
- Trata a todos los procesos de forma igualitaria, sin diferenciar por prioridad. Evita la inanición (*starvation*) por falta de recursos y garantiza que ningún proceso monopolice el procesador.



Introducción

Round-robin scheduling

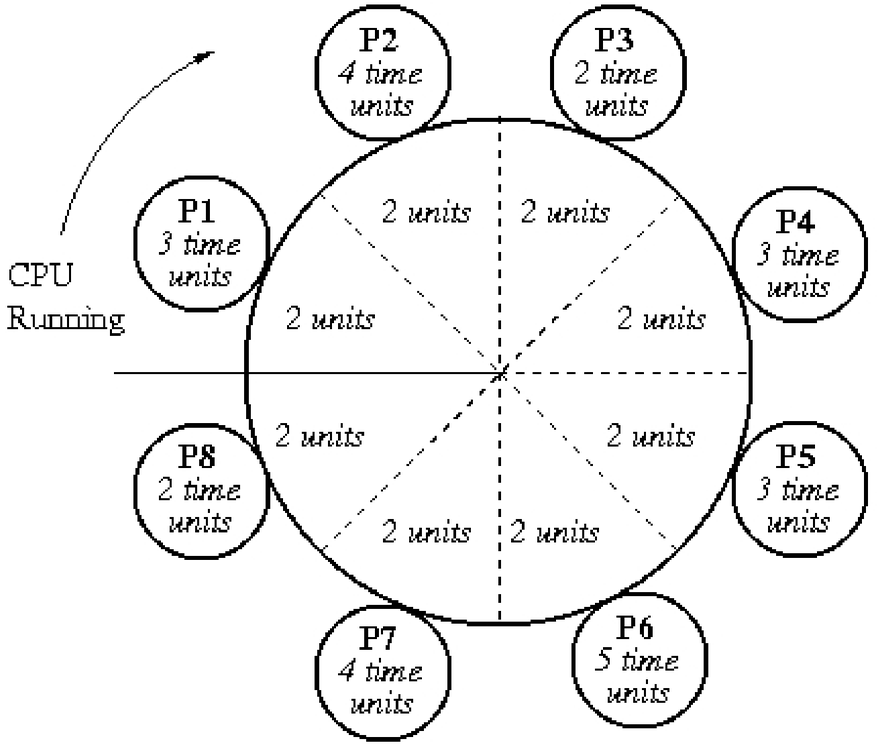
- Se selecciona un proceso en cola.
- Si el tiempo de ejecución restante (*Burst time)* es menor que el *quantum*, se ejecuta hasta completarse y finalizar. Sale del ciclo.
- Else, se ejecuta solo durante el *quantum* asignado. Si no ha terminado, es interrumpido y regresa al final de la cola. Recibirá otro quantum de ejecución en el siguiente ciclo.



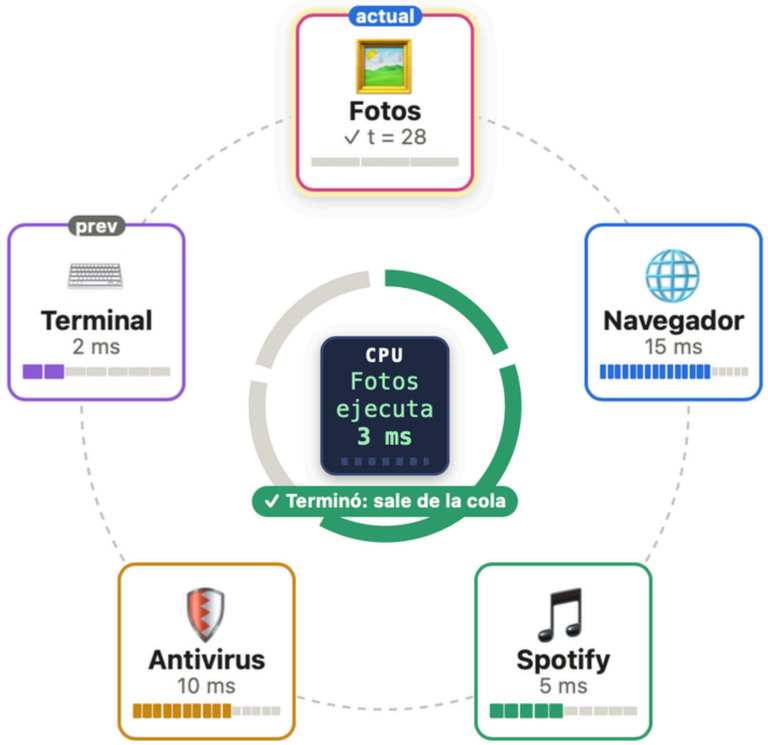
Introducción

Round-robin scheduling

- Se puede implementar mediante lista enlazada circular.
- Si el tamaño del quantum es demasiado grande, se comporta como *First-Come, First-Served (FCFS).* Si es demasiado pequeño, se gasta demasiado cambiando entre procesos (*context-switching overhead)*, degradando su rendimiento.
- **Aplicaciones.** Windows y Linux (SCHED_RR) lo usan para repartir ráfagas de tiempo entre hilos con la misma prioridad. Balanceadores de carga en redes y servidores.



Animación


actual
Fotos
✓ t = 28
prev
Terminal
2 ms
Navegador
15 ms
CPU
Fotos
ejecuta
3 ms
✓ Terminó: sale de la cola
Antivirus
10 ms
Spotify
5 ms

Casos Borde

Lista vacía al comenzar

```
if (ultimo == null) return;
```

Si se recibe una lista sin procesos, se detiene el algoritmo para evitar error *NullPointerException.*

Último proceso en lista

```
if (actual.sig == actual) break;
```

Detecta cuando queda el último proceso en la lista circular para romper el *while (true)* y finalizar. De lo contrario, el while (true) nunca terminaría y volvería a "terminar" siempre el mismo proceso

Quantum inválido

Se asume por definición que el tiempo Quantum es siempre positivo *q > 0.* De lo contrario el reloj no avanzaría, los procesos no reducirían su tiempo y el programa correría indefinidamente.

Complejidad

Complejidad temporal

Variables del sistema a considerar:

N: El número de procesos a ejecutar.

q: El quantum o intervalo temporal límite por turno.

T: El tiempo total de todos los procesos.

actual.resta: El tiempo de ráfaga, o tiempo de duración de un solo proceso.

Complejidad

Complejidad temporal.

Mejor de los casos:

La complejidad es O(N).
Sucede en los casos donde todo proceso a ejecutar cumple **tiempo del proceso ≤ quantum.**

Si todos los procesos duraran ≤ q (por ejemplo, q = 20), el bucle while se ejecutaría N veces. **(se ejecuta el else)**

```
while (true) {
    int usa = Math.min(q, actual.resta);
    actual.resta -= usa;
    reloj += usa;
    if (actual.resta > 0) {
        prev = actual;
        actual = actual.sig;
    } else {
        System.out.println(actual.id + " ter
        if (actual.sig == actual) break;
        prev.sig = actual.sig;
        actual = actual.sig;
    }
}
```

`if (actual.resta > 0)` → Siempre falso

Complejidad

Complejidad temporal.

Peor de los casos y caso general:

La complejidad es O(N + T/q).

¿Por qué N + T/q?
El bucle while se repetirá por proceso:

1) La cantidad de veces que **actual.resta > 0** es verdadero. **(if)**
2) Solo una vez cuando el tiempo que falta del proceso es menor o igual a q **(else).**

```
while (true) {
    int usa = Math.min(q, actual.resta);
    actual.resta -= usa;
    reloj += usa;
    if (actual.resta > 0) {
        prev = actual;
        actual = actual.sig;
    } else {
        System.out.println(actual.id + " ter
        if (actual.sig == actual) break;
        prev.sig = actual.sig;
        actual = actual.sig;
    }
}
```

Complejidad

¿Por qué N+T/q?

- Expresando **$T=T_1+T_2+T_3+...+T_n$. (60ms=20ms+10ms+5ms+15ms+7ms+3ms)**
- Donde **T_i** es la duración del proceso **i**. **(Ej: 20 ms dura ejecutándose el navegador)**
- El proceso usa $\lfloor$**Ti/q**$\rfloor$ turnos completos y, si le sobra tiempo, un turno final más corto. **(Ej: Antivirus: 15/5 = 3 turnos. Terminal: 7/5 → 1 turno completo + 1 parcial = 2 turnos)**
- Así, por cada proceso el bucle se ejecuta como **máximo 1+$\lfloor$Ti/q$\rfloor$ veces**, y para los N procesos que están en la cola se tendrá:

$$(1+\lfloor T_1/q \rfloor)+(1+\lfloor T_2/q \rfloor)+...+(1+\lfloor T_N/q \rfloor) \leq (1+T_1/q)+(1+T_2/q)+...+(1+T_N/q) = N+(T_1+T_2+...+T_N)/q$$

$$N+(T_1+T_2+...+T_N)/q = \mathbf{N+T/q}$$

Complejidad

Complejidad Espacial Auxiliar.

- **Complejidad O(1)** (Espacio Constante)

- **Sustentación del Código:**
  - Uso de memoria In-Place (en el lugar). El método no crea arrays, listas adicionales, ni clona la estructura recibida.
  - Solo reserva espacio para 4 variables de control: prev, actual, reloj y usa.

Complejidad

Complejidad Espacial Total.

- **Complejidad O(N)** (Espacio Lineal)

- **Sustentación del Código:**
  - El sistema total hace uso de una lista circular con N nodos.
  - Aunque cada nodo cuenta con varios atributos, estos son siempre constantes.

Complejidad

Conclusión de complejidades:

	Complejidad
Tiempo, mejor caso	**O(N)**
Tiempo, peor caso y caso general	**O(N + T/q)**
Espacio auxiliar	**O(1)**
Espacio total	**O(N)**

}→ El tiempo depende de N, T y q

}→ El espacio solo depende de N

Créditos

Información:

https://www.ibm.com/docs/es/aix/7.1.0?topic=options-round-robin-distribution

https://www.geeksforgeeks.org/dsa/round-robin-scheduling-with-different-arrival-times/

https://www.ibm.com/docs/en/datapower-gateway/10.6.0?topic=groups-algorithms-making-load-distribution-decisions

https://ieeexplore.ieee.org/document/8862609

https://www.youtube.com/watch?v=YNBKR8eQJnE

https://gemini.google.com/app

Creación de la animación:

https://claude.ai